

RAG 문서준비와 저장

긴 세제개편 자료를 검색할 수 있는 상태로 만들기까지

250쪽짜리 세제개편안 상세본을 어떻게 검색할 수 있는 상태로 준비할까요?

문서 준비 = 질문이 오기 전에, 세제개편안 자료를 찾기 좋은 상태로 미리 정리해 두는 일

지난 시간(01)에 한 칸으로만 보았던 「문서 준비(Indexing)」 안으로 들어갑니다.

2026년 세제개편안 상세본 한 건이 검색용 저장소가 되기까지를 다섯 단계로 따라갑니다.

오늘의 질문과 배울 순서

질문은 지난 시간과 같습니다. 다만 오늘은 그 질문에 답하지 않고, 답할 수 있도록 준비합니다.

**250쪽짜리 세제개편안 상세본을
어떻게 검색할 수 있는 상태로 준비할까요?**

01에서 정한 질문 (그대로)

“2026년 세제개편안에서 신설되는
국내생산세액공제는 언제부터 적용되니까?”

오늘은 이 질문에 답하지 않습니다. 나중에 답할 수 있도록 문서를 준비합니다.

① 읽기

세제개편안의 글자와 구조를 읽
어 들입니다

② 나누기

정책의 의미가 끊기지 않도록
조각(Chunk)으로 나눕니다

③ 꼬리표

조각마다 문서·정책·출처를 연
결합니다

④ 숫자로

조각의 의미를 숫자(Vector)로
바꿉니다

⑤ 저장

셋을 연결해 검색용 저장소에
담습니다

③ 꼬리표와 ④ 숫자로는 순서대로 하는 일이 아니라, 하나의 Chunk에서 갈라지는 두 가지 준비 작업입니다.

오늘 배울 것은 「찾는 방법」이 아니라 「찾을 수 있게 만들어 두는 방법」입니다.

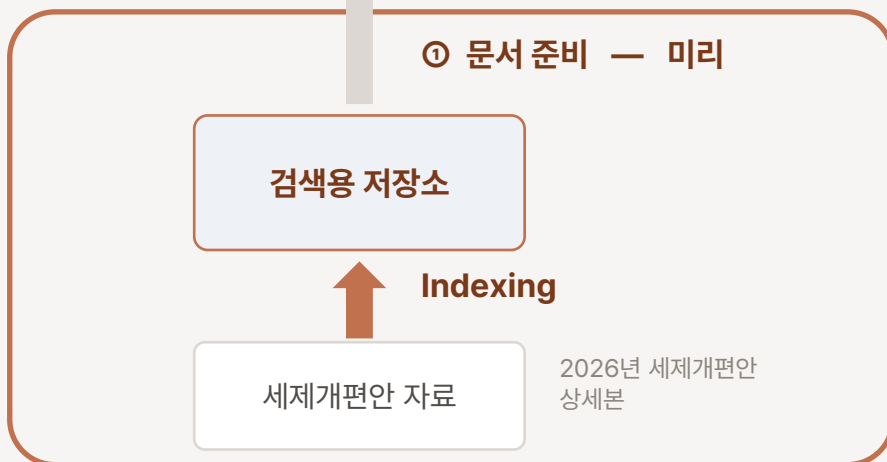
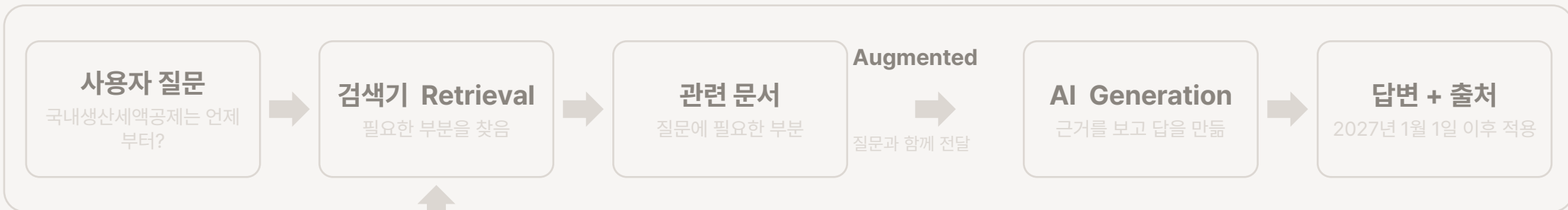
다음 → 지난 시간에 그린 그림에서, 오늘은 어느 부분을 여는 걸까요?

Chunk 에서 갈라지는 두 갈래

01에서 여기까지 왔습니다

지난 시간 마지막에 본 그림입니다. 오늘은 이 그림의 아래쪽 「문서 준비」 부분만 확대합니다.

② 질문 처리 — 질문이 들어올 때마다



오늘(02-1) 여는 부분과, 다음 시간에 여는 부분

02-1 오늘 — 검색용 저장소를 어떻게 만드는가

왼쪽 아래 「세계개편안 자료 → Indexing → 검색용 저장소」 구간을 다섯 단계로 확대합니다.

02-2 다음 — 질문이 들어왔을 때 어떻게 찾는가

위쪽 흐린 부분 중 「사용자 질문 → 검색기 → 관련 문서」 구간을 다룹니다.

03 그 다음 — 찾아온 자료로 어떻게 답하게 하는가

「Augmented → AI → 답변 + 출처」 구간을 다룹니다.

01 25쪽에서 “검색할 문서를 모아 둔 곳 — 만드는 방법과 정식 이름은 뒤에서 봅니다”라고 미뤄 두었던 부분입니다.

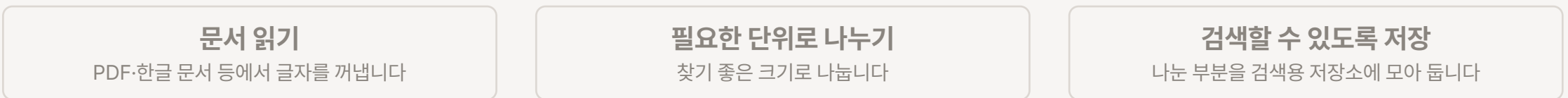
다음 → 01에서는 이 부분을 “세 가지 일”이라고만 말했습니다. 실제로는 몇 단계일까요?

Chunk 에서 갈라지는 두 갈래

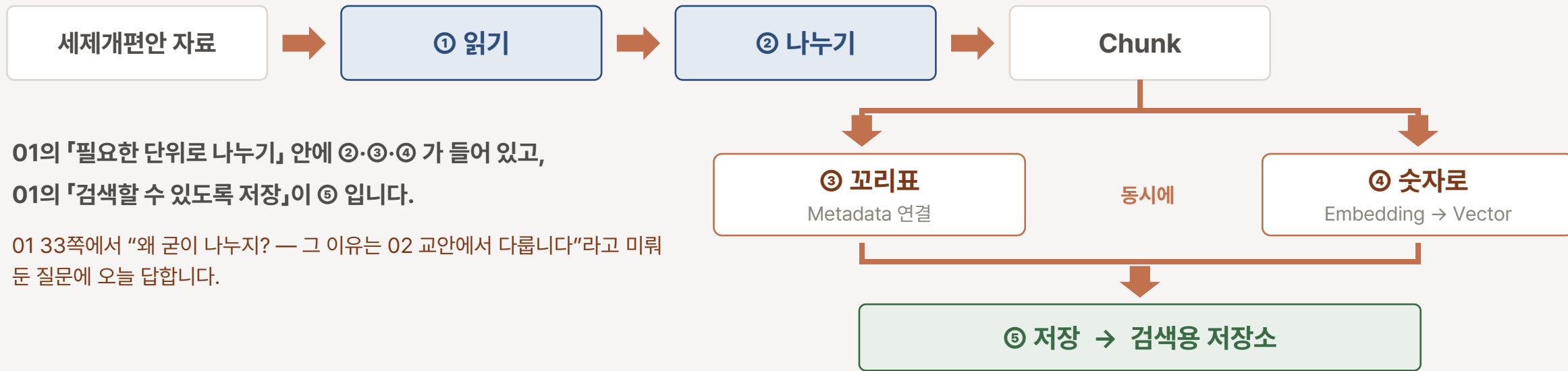
01의 '세 가지 일'을 열어 봅니다

01 33쪽에서는 문서 준비를 세 가지 일로 소개했습니다. 그 안을 열면 다섯 단계가 됩니다.

01 33쪽에서 배운 세 가지 일



열어 보면 —



01의 「필요한 단위로 나누기」 안에 ②·③·④가 들어 있고,
01의 「검색할 수 있도록 저장」이 ⑤입니다.

01 33쪽에서 “왜 굳이 나누지? — 그 이유는 02 교안에서 다룹니다”라고 미뤄
둔 질문에 오늘 답합니다.

다음 → 먼저 ① 읽기부터. 사람이 보는 이 문서를 컴퓨터는 어떤 모습으로 읽을까요?

Chunk 에서 갈라지는 두 갈래

사람이 보는 문서, 컴퓨터가 읽는 문서

우리는 화면에 보이는 모양을 봅니다. 컴퓨터는 그 모양이 아니라 글자와 구조를 읽어 들입니다.

사람이 보는 것

세제개편안 상세본

I. 잠재성장률 반등 지원

(1) 국내생산세액공제 신설

① 요건 및 적용기한(조특법 §29①)

<개정이유> 국내생산 지원

현 행	개 정 안	<적용시기>
<신 설>	요건·적용기한 ...	'27.1.1. ~

눈으로 보고 이해합니다



읽어 들이기

컴퓨터가 다루는 것

글자

본문에 적힌 모든 문자

문서 구조

장 · 절 · 정책 · 현행/개정안 표 · 적용시기

글자만 남기면 "'27.1.1.'" 같은 날짜는 남지만,
그 날짜가 어느 정책의 어떤 날짜인지는 사라집니다.

그래서 문서를 읽을 때 글자와 함께 구조도 남겨 두어야 합니다.

컴퓨터는 화면을 보는 것이 아니라, 글자와 구조를 읽어 들입니다.

다음 → 그렇게 읽어 들인 결과는 실제로 어떻게 생겼을까요?

Chunk 에서 갈라지는 두 갈래

꺼내 보면 이런 모습입니다

2026년 세제개편안 상세본에서 실제로 읽어 들인 부분입니다. (필요한 곳만 잘라 보았습니다)

I. 잠재성장을 반등 지원

1 미래성장동력 확충

(1) 국내생산세액공제 신설

① 국내생산세액공제 요건 및 적용기한(조특법 §29①)

[현행] <신 설>

[개정안] □ 요건 및 적용기한 ○ 내국인이 적격품목을 국내에서 직접 생산·판매 ... ○ (적용기한)
'36.12.31.

<개정이유> 국내생산 취약품목의 국내생산 지원

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용

읽어 들인 결과에 남는 구조

장·절

I.~IV. 큰 흐름

정책

(1) 국내생산세액공제 신설

세부항목

① ~ ⑨ — 전체 285개

현행 / 개정안 표

정책 내용의 본체

<개정이유>

개정 배경 한 줄

<적용시기>

언제부터 적용되는가

글자만 꺼내는 것이 아니라, 정책 제목·표·개정이유·적용시기 같은 구조도 함께 남깁니다.

* HWPX 구조 파서가 실제로 읽어 들인 결과에서 필요한 부분만 잘라 옮겼습니다. 출처 : 2026년 세제개편안 상세본

다음 → 그런데 구조까지 남겨야 하는 이유가 무엇일까요?

Chunk 에서 갈라지는 두 갈래

왜 '구조'까지 남겨야 합니까

숫자 하나만 떼어 놓고 보면, 그 숫자가 무엇인지 알 수 없기 때문입니다.

'27.1.1.'

이 날짜만 있으면 무엇인지 알 수 있습니까?

어느 정책입니까?

국내생산세액공제

무엇이 적용됩니까?

생산·판매분부터 적용

시작일입니까, 종료일입니까?

적용시기 — 시작일

근거 조문은 무엇입니까?

조특법 §29①

문서 어디에 있습니까?

(1) 신설 > ① 요건 및 적용기한

이 다섯 가지는 모두 날짜 자체가 아니라, 그 날짜를 둘러싼 정책 제목 · 표 · <적용시기> 표시에 들어 있습니다.

구조를 잃으면 날짜는 남아도 그 날짜의 뜻은 확인하기 어렵습니다.

개발자 참고

실무에서는 표를 텍스트로 옮길 때 [현행]/[개정안] 라벨과 <적용시기> 줄을 정책 본문과 같은 덩어리에 유지하도록 처리합니다. → 부록 A1 · A2

다음 → ① 읽기를 정리하고, 다음 단계로 무엇이 넘어가는지 보겠습니다.

Chunk 에서 갈라지는 두 갈래

① 읽기 — 정리

입력

세제개편안 파일

2026년 세제개편안 상세본
(2026년 8월 3일 발표)

처리

글자와 문서 구조를 함께 읽어 들인다

장 · 절 · 정책 · 표 · 적용시기를 남깁니다

출력

텍스트 + 문서 구조

250쪽 · 약 130,206자 · 장 4개 · 정책 세부항목 285개

지금까지 만들어진 것

2026년 세제개편안 상세본

250쪽 · 표와 구조를 포함한 원본

↓ ① 읽기

텍스트 + 문서 구조

장 · 절 · 정책 · 표 · 적용시기

↓ ② 나누기 (Chunking)

여러 개의 Chunk

③ 꼬리표 (Metadata)

④ 숫자로 (Embedding)

⑤ 저장 → 검색용 저장소

다음 → 읽어 들였습니다. 그런데 250쪽 전체를 통째로 검색 대상으로 둘 수 있을까요?

Chunk 에서 갈라지는 두 갈래

250쪽을 통째로 둘 수 있을까요

문서 한 건을 통째로 하나의 검색 대상으로 두면, 찾아온 결과도 문서 한 건 전체가 됩니다.

2026년 세제개편안 상세본 · 250쪽 · 약 137,416 token



▲ 이 질문에 필요한 근거

나머지는 이 질문의 답과 직접 관계가 없습니다

01에서 확인했듯이, 전체 약 137,416 token 중 이 질문에 직접 답하는 근거는 정책 항목 하나(약 577 token)입니다.

실제 근거 정책 항목 (원문)

“① 국내생산세액공제 요건 및 적용기한(조특법 §29①) ...

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용”

I. 잠재성장률 반등 지원 > (1) 국내생산세액공제 신설 > ①

통째로 두면 생기는 일

- 찾아와도 250쪽이 통째로 옵니다
- 어느 부분이 근거인지 알 수 없습니다
- 어디를 보고 답했는지 말할 수 없습니다

검색은 문서 한 건이 아니라, 답의 근거가 되는 부분을 찾아오는 일이어야 합니다.

다음 → 그렇다면 사람은 이런 두꺼운 보고서에서 어떻게 찾습니까?

Chunk 에서 갈라지는 두 갈래

01에서 이미 답을 보았습니다

01 22쪽에서 확인한 “사람이 찾는 순서”를 뒤집으면, 그대로 “문서를 나누는 기준”이 됩니다.

01 22쪽 — 사람이 찾는 순서

1. 목차를 확인합니다

「I. 잠재성장률 반등 지원」

2. 필요한 쪽을 펼칩니다

「(1) 국내생산세액공제 신설」

3. 그 부분만 읽습니다

「① 요건 및 적용기한」의 <적용시기>



뒤집으면
문서를 나누는 기준

큰 흐름(장) 단위로 I. 잠재성장률 반등 지원

정책 단위로 (1) 국내생산세액공제 신설

정책 세부항목 단위로 ① 제목 + 현행/개정안 표 + <적용시기>

비유

서류철과 같습니다

두꺼운 서류를 한 장으로 묶어 두지 않고, 사안별로 낱장을 나누어 철해 둡니다. 그래야 필요한 낱장만 꺼내 보고, 어디서 꺼냈는지도 말할 수 있습니다.

나누는 기준은 문서가 이미 갖고 있습니다. 목차는 경계를 찾는 기준이고, 실제 조각은 그보다 작은 정책 세부항목 단위가 될 수 있습니다.

다음 → 그렇게 나눈 조각을 무엇이라고 부를까요?

Chunk 에서 갈라지는 두 갈래

그 조각을 Chunk라고 부릅니다

문서를 나눈 하나하나의 조각이 앞으로 검색에서 다루는 기본 단위가 됩니다.

Chunk = 검색할 때 다루게 되는 문서의 한 조각

Chunk 하나의 실제 모습 (세제개편안 상세본 · 약 609자)

① 국내생산세액공제 요건 및 적용기한(조특법 §29①)

[현행] <신 설>

[개정안] □ 요건 및 적용기한 ○ (요건) 내국인이 적격품목을 국내에서 직접 생산·판매

- (국내생산) ①+② - (국내판매) ①~④ 모두 충족 ...

○ (적용기한) '36.12.31.

<개정이유> 국내생산 취약품목의 국내생산 지원

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용

이 조각 하나에 들어 있는 것

- 어느 정책인지 알려 주는 제목과 조문
- 현행과 개정안을 나란히 보여 주는 표
- 개정 배경을 알려 주는 <개정이유>
- 언제부터인지 알려 주는 <적용시기>

이 넷이 한 조각 안에 함께 있어야

“2027년 1월 1일 이후 적용”이라는 답을 뒷받침할 수 있습니다.

* HWPX 구조 파서의 직렬화 결과에서 요약 발췌했습니다. 609자는 문자 수, 약 577 token은 cl100k_base 기준입니다.

좋은 Chunk는 조각만 보아도 무엇에 대한 내용인지 알 수 있고, 가져왔을 때 근거의 의미가 가능한 한 유지되어야 합니다.

다음 → 그럼 그냥 글자 수를 세어 500자씩 자르면 될까요?

Chunk 에서 갈라지는 두 갈래

그냥 500자씩 자르면 될까요

이 정책 단위(609자)를 글자 수만 세어 500자에서 잘라 보았습니다. 실제로 이렇게 갈라집니다.

Chunk 0

① 국내생산세액공제 요건 및 적용기한(조특법 §29①)

[현행] <신 설> [개정안] □ 요건 및 적용기한

○ (요건) 내국인이 적격품목을 국내에서 직접 생산·판매 ...

- (국내판매) ①~④ 모두 충족 ...

③ 특수관계인 거래 확인 서류*를 제출할 것 * 세부 내용은 대통령령

← 여기서 500자를 채워 끊었습니다

Chunk 1

에서 규정

④ 중고품이 아닐 것

○ (적용기한) '36.12.31.

<개정이유> 국내생산 취약품목의 국내생산 지원

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용

답이 되는 <적용시기>는 여기에 있습니다

정책 제목 · 조문 · 요건 대부분은 Chunk 0 에 남고, <적용시기>는 Chunk 1 로 넘어갔습니다.

두 조각은 서로 다른 조각이므로, 검색으로 Chunk 1 만 찾아오면 앞의 설명은 함께 오지 않습니다.

‘언제’는 남았지만 ‘무엇이’는 사라졌습니다.

다음 → 그러면 잘린 조각에는 정확히 무엇이 없는 걸까요?

Chunk 에서 갈라지는 두 갈래

잘린 조각에는 무엇이 없습니까

앞 장의 Chunk 1 을 그대로 두고, 없는 것을 하나씩 확인해 보겠습니다.

○ (적용기한) '36.12.31.

<개정이유> 국내생산 취약품목의 국내생산 지원

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용

✕ “국내생산세액공제”라는 정책 이름

✕ “(조특법 §29①)”이라는 근거 조문

✕ “I. 잠재성장률 반등 지원 > (1) 신설”이라는 상위 경로

구조를 잃으면 어떻게 되는가

이 조각에는 날짜가 두 개 남습니다

'27.1.1. (적용시기 · 시작) — '36.12.31. (적용기한 · 종료)

그런데 정책 이름과 구조가 없으면

어느 정책의 시작·종료인지 확인하기 어렵습니다

날짜는 남아 있어도, 그 날짜가 어느 정책의 어떤 날짜인지 조각만으로는 확인하기 어렵습니다.

시작일과 종료일을 구분해 주는 것이 정책 구조입니다.

구조를 잃으면 남은 정보의 의미를 확인하기 어려워집니다.

※ 출처 : 2026년 세제개편안 상세본 (1) 국내생산세액공제 신설 ① (논리 p.1)

다음 → 그러면 반대로 더 작게 자르면 안전할까요?

Chunk 에서 갈라지는 두 갈래

더 작게 300자로 자르면

같은 정책 단위를 300자마다 잘라 보았습니다. 이번에는 다른 문제가 생깁니다.

Chunk 1

내용은 대통령령에서 규정

- (국내판매) ①~④ 모두 충족

① 공급장소가 국내일 것* ... ④ 중고품이 아닐 것

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·

Chunk 2

판매 분부터 적용

적용시기 문장이 "... 생산·" 과 "판매 분부터 적용" 으로 갈라졌습니다.

마지막 조각(Chunk 2)에는 9자짜리 "판매 분부터 적용"만 남습니다.

이 조각만 보면 사람이 보아도 무엇에 대한 내용인지 알 수 없습니다.

너무 작게 자르면 문장과 조건 목록까지 잘립니다.

다음 → 그럼 조각을 조금씩 겹쳐서 자르면 해결될까요?

겹쳐서 자르면 해결될까요

앞 조각의 끝부분을 다음 조각의 앞에 겹쳐 넣는 방법을 겹침(overlap)이라고 합니다.

같은 정책 단위를 500자마다 자르되, 앞 조각의 마지막 100자를 다음 조각 앞에 겹쳐 넣었습니다.

겹쳐서 자른 Chunk 1 의 실제 앞부분

- ② 생산된 과세연도 또는 다음 과세연도에 판매될 것
- ③ 특수관계인 거래 확인 서류 제출 ④ 중고품이 아닐 것
- (적용기한) '36.12.31. <개정이유> 국내생산 지원
- <적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용

겹침으로 되살아난 것

- 국내판매 요건 일부(②~)
- 앞 조각 끝과 이어지는 문장

그래도 여전히 없는 것

- × 정책 제목
- × 근거 조문(\$29①)
- × 상위 경로((1) 신설)

겹침은 일부 문맥을 살리지만 만능은 아닙니다.

개발자 참고

겹침 크기를 키우면 되살아나는 범위는 넓어지지만 저장량과 중복이 함께 늘어납니다. 글자 수 기준 겹침만으로는 정책 제목·조문이 함께 온다고 보장할 수 없습니다. → 부록 A1·A2

다음 → 그렇다면 몇 자로 자르는 것이 정답일까요?

Chunk 에서 갈라지는 두 갈래

글자 수만으로는 부족합니다

지금까지 본 실패는 모두 “글자 수만 세어 잘랐기 때문에” 생긴 일입니다.

너무 작게 자르면

- 문장과 표가 도중에 끊깁니다
- 날짜와 조건 목록이 잘릴 수 있습니다
- 조각만 보면 무슨 말인지 알 수 없습니다

300자 사례

너무 크게 자르면

- 관련 없는 내용까지 함께 들어옵니다
- 한 조각에 여러 주제가 섞입니다
- 어느 부분이 근거인지 가리기 어렵습니다

문서 전체를 한 조각으로 둔 경우

그래서

- 글자 수만 기준으로 삼지 않습니다
- 문서의 구조를 함께 봅니다
- 의미가 끊기지 않는 지점에서 나눕니다

다음 장에서 이어집니다

“몇 자가 정답이다” 라고 정해진 숫자는 없습니다. 문서의 종류와 내용에 따라 달라집니다.

개발자 참고

실무에서는 문서 종류별로 조각 크기와 겹침 크기를 정해 두고, 검색 결과를 보면서 조정합니다. 구체적인 권장 범위와 도구별 설정값은 부록 A1 에서 다룹니다.

다음 → 그럼 무엇을 기준으로 나누어야 할까요?

Chunk 에서 갈라지는 두 갈래

문서의 구조를 따라 나눕니다

① 읽기에서 남겨 둔 정책 제목 · 표 · 개정이유 · 적용시기가 여기서 나누는 기준이 됩니다.

①

정책 세부항목 제목을 경계로 나눕니다

「① 국내생산세액공제 요건 및 적용기한」 같은 제목이 경계가 됩니다.
세제개편안 상세본은 이런 세부항목을 285개 갖고 있습니다.

②

현행/개정안 표는 통째로 둡니다

[현행]과 [개정안]의 대응 관계가 끊기지 않게 같은 조각 안에 둡니다.
표 한가운대를 자르지 않습니다.

③

<개정이유>·<적용시기>를 떼어 놓지 않습니다

정책 단위 맨 끝에 붙는 이 두 줄이 다른 조각으로 넘어가지 않게 합니다.
날짜와 그 날짜를 설명하는 정책 제목을 떼어 놓지 않습니다.

그 결과 만들어지는 조각

① 국내생산세액공제 요건 및 적용기한(조특
법 §29①)

[현행] <신 설> [개정안] □ 요건 및 적용기
한 ...

<개정이유> 국내생산 취약품목의 국내생산
지원

<적용시기> '27.1.1. 이후 ... 적용

제목 · 표 · 개정이유 · 적용시기가 한 조각 안
에 함께 있습니다.

나누는 기준은 글자 수가 아니라 문서의 구조입니다. 이 문서에서는 정책 세부항목이 자연스러운 Chunk 후보가 됩니다.

개발자 참고

문자 수 기준 분할, 구분자 기준 분할, 제목 기준 분할, 의미 기준 분할 등 여러 방식이 있으며 함께 쓰기도 합니다. → 부록 A1

다음 → 나누었습니다. 다음 단계로 무엇이 넘어가는지 정리하겠습니다.

Chunk 에서 갈라지는 두 갈래

② 나누기 (Chunking) — 정리

입력

텍스트 + 문서 구조

① 읽기에서 넘어온 약 130,206자와
정책 제목 · 표 · 개정이유 · 적용시기 정보

처리

문맥이 유지되도록 문서 조각으로 나눈다

정책 제목 단위 → 표는 통째로 → 개정이유·적용시기 함께 두기

출력

여러 개의 Chunk

조각만 보아도 무엇에 대한 내용인지 알 수 있는 상태

지금까지 만들어진 것

2026년 세제개편안 상세본

250쪽 · 표와 구조를 포함한 원본

↓ ① 읽기

텍스트 + 문서 구조

장 · 절 · 정책 · 표 · 적용시기

↓ ② 나누기 (Chunking)

여러 개의 Chunk

문맥이 유지되는 문서 조각

③ 꼬리표 (Metadata)

④ 숫자로 (Embedding)

⑤ 저장 → 검색용 저장소

다음 → 조각으로 나눴습니다. 그런데 이 조각만 보면 어떤 문서의 어떤 정책인지 알 수 있을까요?

Chunk 에서 갈라지는 두 갈래

이 조각, 어떤 문서의 어떤 정책입니까

잘 나눈 조각이라도, 조각 하나만 따로 놓고 보면 여전히 알 수 없는 것이 있습니다.

17쪽에서 구조를 따라 잘 나눈 Chunk

① 요건 및 적용기한(조특법 §29①)

[개정안] □ 요건 및 적용기한 ...

<개정이유> 국내생산 취약품목의 국내생산 지원

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용

이 조각 안에 답은 들어 있습니다. 그런데 —

어느 자료의 조각입니까?

조각 안에 “2026년 세제개편안”이라는 말이 없습니다

어느 문서에서 나왔습니까?

상세본인지 문답자료인지 알 수 없습니다

어느 파일에서 나왔습니까?

원본을 다시 확인하려 해도 찾아갈 수 없습니다

왜 지금 문제가 되는가

검색용 저장소에는 이 조각 하나만 들어가는 것이 아닙니다.

문답자료 · 개조식, 그리고 서로 다른 정책 285개의 조각이 함께 들어갑니다.

모두 “세액공제”, “<적용시기>” 같은 비슷한 말을 씁니다.

조각의 신원이 없으면, 나중에 답을 찾아와도 그것이 어느 문서의 어느 정책 내용인지 말할 수 없습니다.

잘 나눈 조각에도 “이것이 무엇의 조각인가”는 적혀 있지 않습니다.

다음 → 사람은 이럴 때 무엇을 합니까?

조각에 꼬리표를 붙입니다 — Metadata

사무실에서 서류에 접수인과 분류번호를 붙이는 것과 같은 일을 문서 조각에도 합니다.

비유

서류의 접수인과 같습니다

서류 내용은 그대로 두고, 겉면에 접수일 · 담당부서 · 문서번호를 따로 붙입니다. 내용을 고치지 않고도 그 서류가 무엇인지 알 수 있게 됩니다.



문서 조각에 이렇게 붙이는 정보를
Metadata (메타데이터) 라고 부릅니다

이번 교안에서 사용할 꼬리표 — 세 묶음

① 어떤 자료인가

문서명 · 자료 종류 · 발표일

② 어디에서 나온 내용인가

장 · 절 · 정책 경로

③ 원본으로 어떻게 돌아가는가

원본 파일 · 조각 번호

**document_title · document_type ·
publication_date**

①의 실제 필드명 (개발자 참고)

section_path

②의 실제 필드명

source_file · chunk_id

③의 실제 필드명

Metadata는 조각의 내용이 아니라, 조각의 신원과 출처를 적어 두는 정보입니다.

다음 → 실제 세제개편안 조각에 붙이면 어떤 값이 들어갈까요?

Chunk 에서 갈라지는 두 갈래

세제개편안 조각의 실제 꼬리표

앞에서 만든 Chunk 하나에 꼬리표 값을 실제로 채워 넣으면 다음과 같습니다.

Chunk 원문

① 요건 및 적용기한(조특법 §29①)

[현행] <신 설> [개정안] □ 요건 및 적용기한 ...

<개정이유> 국내생산 취약품목의 국내생산 지원

<적용시기> '27.1.1. 이후 개시하는 과세연도 생산·판매 분부터 적용



연결

Metadata

document_title = 2026년 세제개편안

document_type = 상세본

publication_date = 2026-08-03

section_path = I. 잠재성장을 반등 지원 > 1. 미래성장동력 확충 > (1) 국내생산세액공제 신설 > ①

source_file = 2. 2026년 세제개편안 상세본.hwp

chunk_id = TXP26-7ae2a642a652-0

01 36쪽에서 답과 함께 「2026년 세제개편안 상세본 / I. 잠재성장을 반등 지원 > (1) 국내생산세액공제 신설 > ① 요건 및 적용기한」이라는 출처를 보았습니다.

나중에 이런 출처를 표시하려면, 원본 파일과 정책 경로를 지금 이렇게 Metadata로 미리 연결해 두어야 합니다.

Metadata는 조각의 신원을 잃지 않게 하고, 나중에 원본으로 되돌아갈 길을 남깁니다.

다음 → 그런데 정책 이름만 있으면 충분할까요?

Chunk 에서 갈라지는 두 갈래

문답자료와 나란히 놓으면

두 조각 모두 같은 정책, 「국내생산세액공제」를 다루고 있습니다.

상세본

document_type = 상세본

section_path = (1) 국내생산세액공제 신설 > ①

page (논리) = p.1

요건·적용기한을 규정하는 본문

<적용시기> '27.1.1. 이후 적용

문답자료

document_type = 문답자료

section_path = (1) 국내생산세액공제 신설 (상세본 p.1)

page (논리) = p.1

신설 취지를 설명하는 Q&A

“녹색전환·경제안보 차원에서 도입”

정책 이름만 맞으면 충분할까요?

같은 정책이라도 어느 문서의 몇 쪽에서 나왔는지 함께 남겨 두어야 합니다.

* 두 문서 모두 논리 p.1이어서 문서 구분이 없으면 페이지 번호도 겹칩니다. 문답자료 원문에는 「(상세본 p.1)」 상호참조가 실제로 적혀 있습니다.

다음 → 신원은 남겼습니다. 그런데 나중에 질문과 비교하려면 무엇을 준비해야 할까요?

Chunk 에서 갈라지는 두 갈래

③ 꼬리표 (Metadata) — 정리

입력

Chunk

② 나누기에서 넘어온 문서 조각 하나

처리

자료 · 문서 종류 · 정책 경로 · 원본 · 조각번호를 연결한다

조각의 내용은 고치지 않고, 신원 정보를 따로 붙입니다

출력

Chunk + Metadata

조각 하나가 "어느 문서의 어느 정책인지" 말할 수 있는 상태

지금까지 만들어진 것

2026년 세제개편안 상세본

250쪽 · 표와 구조를 포함한 원본

↓ ① 읽기

텍스트 + 문서 구조

장 · 절 · 정책 · 표 · 적용시기

↓ ② 나누기 (Chunking)

여러 개의 Chunk

문맥이 유지되는 문서 조각

③ 꼬리표 (Metadata)

문서 · 정책 경로 · 출처

동시에

④ 숫자로 (Embedding)

⑤ 저장 → 검색용 저장소

다음 → 신원은 확인했습니다. 그런데 나중에 질문과 비교하려면 무엇을 준비해야 할까요?

Chunk 에서 갈라지는 두 갈래

나중에 질문과 비교하려면 무엇을 준비해야 할까요

지금은 아직 질문이 오기 전입니다. 조각은 이미 수천 개가 만들어져 있습니다.

지금 상황

Chunk 1

Chunk 2

Chunk 3

Chunk 4

Chunk 5

Chunk 6

... 이런 조각이 상세본 한 건에서만 285개, 문답자료·개조식까지 더하면 수백 개가 됩니다.

사람이라면

조각을 하나씩 읽어 보고

질문과 관련 있는지 판단합니다

그런데 컴퓨터가 수천 개를

질문이 들어올 때마다 하나씩

읽어 보는 것은 현실적이지 않습니다

그래서

질문이 오기 전에 미리

“비교할 수 있는 형태”로 바꿔 둡니다

질문이 오기 전에, 조각을 미리 비교할 수 있는 형태로 바꿔 둡니다.

※ 실제로 어떻게 비교하고, 그중 몇 개를 골라오는지는 다음 교안(02-2)에서 다룹니다. 오늘은 “비교할 수 있게 준비해 두는 것”까지만 봅니다.

다음 → 그 “비교할 수 있는 형태”가 무엇일까요?

Chunk 에서 갈라지는 두 갈래

Day 2에서 배운 것을 여기서 씁니다

둘째 날에 “문장을 숫자로 바꾼다”는 것을 이미 보았습니다. 그 일이 지금 필요한 준비입니다.

Day 2 에서 배운 것

문장 → 숫자

글로 된 문장을 컴퓨터가 다룰 수 있는 숫자로 바꿉니다.



02-1 에서 하는 일

Chunk 텍스트 → 숫자

문서 조각의 의미를 숫자로 바꿔서 저장해 둡니다.

문서 조각의 의미를 숫자로 바꾸는 이 과정을 Embedding (임베딩) 이라고 부릅니다.

왜 숫자로 바꿉니까?

나중에 사용자 질문과 문서 조각의 의미를 비교할 수 있도록 미리 준비하기 위해서입니다.

숫자로 바꾸는 것 자체가 목적이 아닙니다. Embedding은 검색을 위한 준비 작업입니다.

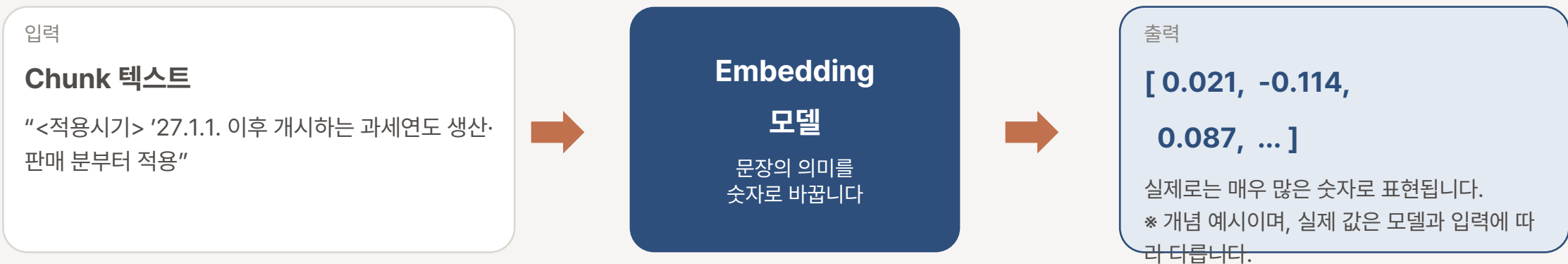
숫자로 바꾸는 이유는, 나중에 의미를 비교하기 위해서입니다.

다음 → 실제 세제개편안 조각을 바꿔 보면 어떻게 될까요?

Chunk 에서 갈라지는 두 갈래

세제개편안 조각을 숫자로 바꾸면

앞에서 만든 Chunk 의 텍스트를 Embedding 모델에 넣으면 숫자 묶음이 나옵니다.



이렇게 만들어진 숫자 묶음을 **Vector (벡터)** 라고 부릅니다.

지금 기억할 것은 두 가지뿐입니다.

- **Chunk 텍스트를 넣으면 Vector 가 나온다**
- **그 Vector 는 나중에 질문과 비교하기 위한 것이다**

개발자 참고

실제 숫자의 개수(차원)는 사용하는 모델에 따라 다릅니다. 모델명과 차원, 문서용·질문용 처리 구분은 부록 A3 에서 다룹니다.

Chunk 텍스트를 넣으면 Vector 가 나옵니다.

다음 → 그러면 앞에서 붙인 꼬리표도 함께 숫자로 바뀌는 걸까요?

Chunk 에서 갈라지는 두 갈래

꼬리표와 Vector는 서로 다른 역할을 합니다

③ 꼬리표와 ④ 숫자로는 순서대로 하는 일이 아니라, 하나의 Chunk 에서 갈라지는 두 가지 준비입니다.



Embedding 모델에 넣는 것은 Chunk 텍스트입니다.

문서명 · 정책 경로 · 파일명 · 조각번호 같은 꼬리표는 별도로 연결해 관리합니다.

Metadata 의 역할

- 이 조각이 무엇의 어느 부분인지 밝힙니다
- 원본 파일로 되돌아갈 길을 남깁니다
- 문서 종류 · 정책 경로로 구분하게 해 줍니다

Vector 의 역할

- 조각의 내용이 어떤 의미인지 담습니다
- 나중에 질문과 의미를 비교할 수 있게 합니다
- 내용이 비슷한 조각을 가려낼 수 있게 합니다

개발자 참고

Section 제목처럼 문맥을 알려 주는 일부 정보를 Chunk 텍스트 앞에 함께 넣어 Embedding 하는 구현 방식도 있습니다. → 부록 A2

Metadata는 조각의 신원을, Vector는 조각의 의미를 말합니다.

다음 → ④ 숫자로를 정리하고 마지막 단계로 가겠습니다.

Chunk 에서 갈라지는 두 갈래

④ 숫자로 (Embedding) — 정리

입력

Chunk 텍스트

② 나누기에서 만든 조각의 본문
(꼬리표는 여기에 들어가지 않습니다)

처리

Embedding 모델이 문장의 의미를 숫자로 바꾼다

나중에 질문과 비교할 수 있게 하기 위한 준비입니다

출력

Vector

[0.021, -0.114, 0.087, ...] 실제로는 매우 많은 숫자 (개념 예시)

지금까지 만들어진 것

2026년 세제개편안 상세본

250쪽 · 표와 구조를 포함한 원본

↓ ① 읽기

텍스트 + 문서 구조

장 · 절 · 정책 · 표 · 적용시기

↓ ② 나누기 (Chunking)

여러 개의 Chunk

문맥이 유지되는 문서 조각

③ 꼬리표 (Metadata)

문서 · 정책 경로 · 출처

동시에

④ 숫자로
(Embedding)

Chunk 텍스트 → Vector

⑤ 저장 → 검색용 저장소

다음 → 숫자는 만들었습니다. 그런데 Vector만 저장하면 원문과 출처는 어떻게 확인할까요?

Chunk 에서 갈라지는 두 갈래

Vector만 저장하면 될까요

숫자만 남겨 두면, 나중에 사람에게 보여 줄 것이 남지 않습니다.

Vector 만 저장한 경우

[0.021, -0.114, 0.087, ...]

- ✗ 조각의 원문을 사람에게 보여 줄 수 없습니다
- ✗ 어느 문서의 어떤 정책인지 말할 수 없습니다
- ✗ 원본 파일로 되돌아갈 수 없습니다

근거를 확인해야 하는 업무에서는 특히 중요합니다

숫자 하나를 근거로 판단하려면, 그 숫자가 적힌 원문과 출처를 함께 확인할 수 있어야 합니다.

01 38쪽에서 “출처는 자동으로 붙지 않습니다 — 미리 만들어 두어야 표시됩니다” 라고 배웠습니다.

그 “미리 만들어 두는 일”이 바로 이 단계입니다.

숫자만 남기면 근거로 쓸 수 없습니다. 원문과 신원을 함께 남겨야 합니다.

다음 → 그러면 무엇을 함께 보관해야 할까요?

Chunk 에서 갈라지는 두 갈래

세 가지가 함께 연결됩니다

조각 하나를 저장할 때, 원문 · Vector · Metadata 셋이 서로 연결된 상태로 보관됩니다.

Chunk 원문

"① 국내생산세액공제 요건 및 적용기한 ...

<적용시기> '27.1.1. 이후 ... 적용"

사람에게 근거로 보여 줄 실제 문장

+

Vector

[0.021, -0.114, 0.087, ...]

질문과 의미를 비교하기 위한 숫자 (개념 예시)

+

Metadata

document_type = 상세본

publication_date = 2026-08-03

section_path = l. ... > (1) 신설 > ①

이 조각의 신원과 출처



이 셋이 서로 연결된 한 묶음이 저장소에 들어가는 한 건이 됩니다.

원문 · Vector · Metadata 가 서로 연결되어 있어야 합니다.

다음 → 이렇게 저장하고 관리하는 곳을 무엇이라고 부를까요?

Chunk 에서 갈라지는 두 갈래

이것이 01에서 말한 '검색용 저장소'입니다

01 30쪽에서 이름을 붙였고, 25쪽에서 “정식 이름은 뒤에서 봅니다”라고 미뤄 두었던 그것입니다.

검색용 저장소는 문서 조각을 Vector를 이용해 찾을 수 있도록 저장하고 관리하는 곳입니다.

검색용 저장소		
Chunk 원문	Vector	Metadata
① 국내생산세액공제 요건·적용기한 ... <적용시기> '27.1.1. ...	[0.021, -0.114, ...]	상세본 / (1) 국내생산세액공제
(2) 친환경 승용차 감가상각 한도 ... 800만원 → 1,000/700만원 ...	[0.008, 0.191, ...]	상세본 / (2) 친환경 승용차
(1) 국내생산세액공제 신설 취지 ... 녹색전환·경제안보 ...	[0.037, -0.052, ...]	문답자료 / (1) 국내생산세액공제

이름에 대하여

검색용 저장소 (Vector DB)

기술 문서에서는 Vector DB 라고 부릅니다.

이 교안에서는 01과 같이 「검색용 저장소」라는 이름을 계속 사용합니다.

※ 이해를 돕기 위한 그림이며 실제 저장 방식과 동일하지 않습니다. 1·2행은 같은 문서의 다른 정책, 1·3행은 같은 정책의 다른 문서입니다.

01에서 한 칸으로 그렸던 검색용 저장소의 내부가 바로 이것입니다.

개발자 참고

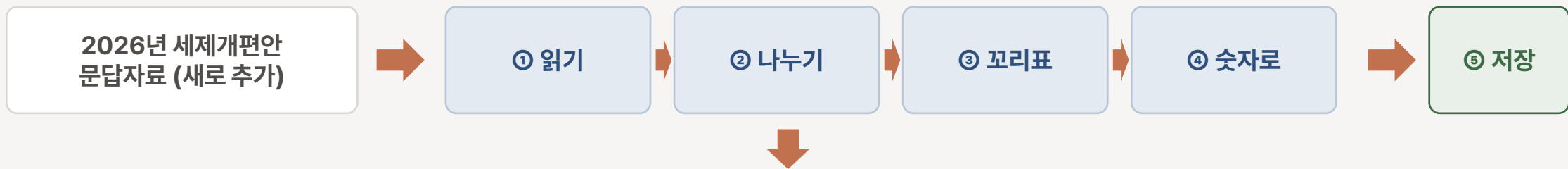
실제 구현체로는 FAISS · Chroma · Qdrant 등이 있으며, 저장 방식과 검색 방식이 서로 다릅니다. → 부록 A4

다음 → 한 번 만들어 두면 끝일까요?

Chunk 에서 갈라지는 두 갈래

새 자료가 들어오면

01 30쪽·38쪽에서 “문서가 바뀌면 다시 정리해야 합니다”라고 배웠습니다. 그 말의 뜻입니다.



새로 들어온 문서를 같은 다섯 단계로 처리해서 검색용 저장소에 추가합니다.

추가되는 경우

상세본만 있었다면, 같은 발표의 문답자료를 추가할 때 그 문서만 ①~⑤를 거칩니다.

갱신되는 경우

기존 문서가 수정·보완되면 해당 문서의 조각만 다시 만들어 바꿔 줍니다.

주의

저장소 전체를 매번 처음부터 다시 만들지는 않습니다. (설계에 따라 다를 수 있음)

문서 준비는 한 번 하고 끝나는 작업이 아니라, 문서가 늘거나 바뀔 때마다 이어지는 작업입니다.

다음 → ⑤ 저장을 정리하겠습니다.

Chunk 에서 갈라지는 두 갈래

⑤ 저장 — 정리

입력

Chunk 원문 + Vector + Metadata

③ 꼬리표와 ④ 숫자로에서 각각 만들어진 것

처리

셋이 서로 연결된 상태로 저장하고 관리한다

Vector를 이용해 조각을 찾을 수 있는 형태로 담아 둡니다

출력

검색용 저장소 (Vector DB)

01에서 한 칸으로 그렸던 바로 그 상자가 완성됩니다

지금까지 만들어진 것

2026년 세제개편안 상세본

250쪽 · 표와 구조를 포함한 원본

① 읽기

텍스트 + 문서 구조

장 · 절 · 정책 · 표 · 적용시기

② 나누기 (Chunking)

여러 개의 Chunk

문맥이 유지되는 문서 조각

③ 꼬리표 (Metadata)

문서 · 정책 경로 · 출처

동시에

④ 숫자로

(Embedding)

Chunk 텍스트 → Vector

⑤ 저장 → 검색용 저장소

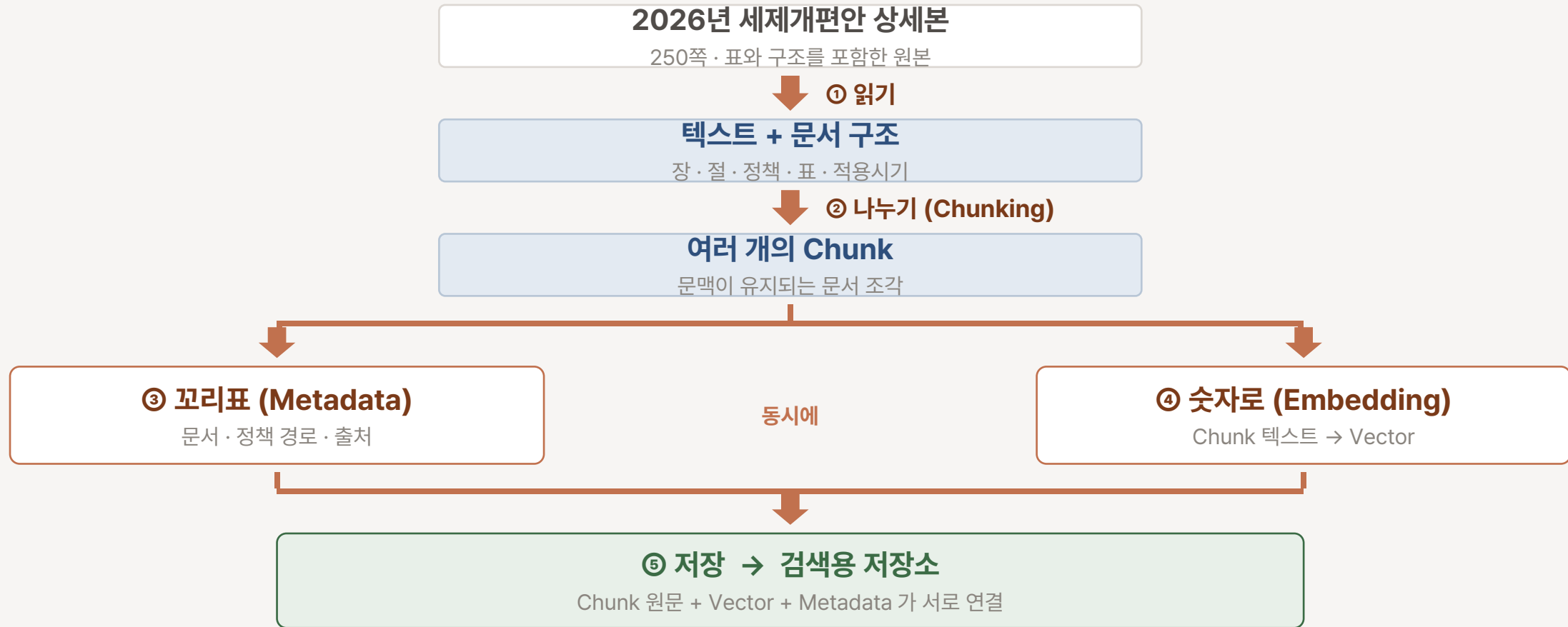
Chunk 원문 + Vector + Metadata 가 서로 연결

다음 → 오늘 배운 다섯 단계를 한 장으로 모아 보겠습니다.

Chunk 에서 갈라지는 두 갈래

오늘 완성한 그림 — 한 장으로

③ 꼬리표와 ④ 숫자로가 하나의 Chunk에서 갈라져 나와, ⑤ 저장에서 다시 만납니다.



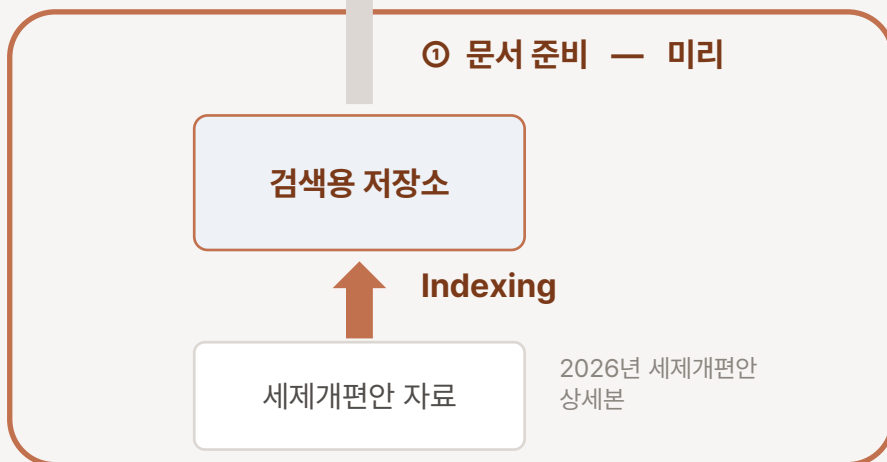
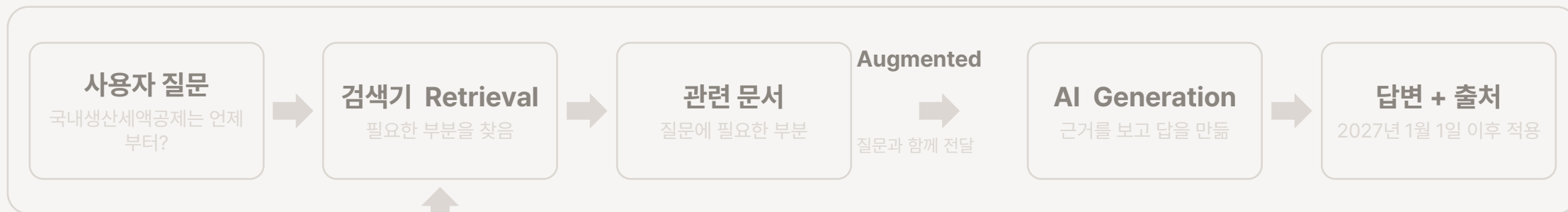
다음 → 이 그림은 01에서 본 그림의 어디에 들어갈까요?

Chunk 에서 갈라지는 두 갈래

01의 그림에 다시 끼워 넣으면

오늘 배운 다섯 단계는 새로운 시스템이 아닙니다. 지난 시간에 한 칸으로 보았던 부분의 내부입니다.

② 질문 처리 — 질문이 들어올 때마다



「세제개편안 자료 → Indexing → 검색용 저장소」 안에서 일어난 일

① 읽기 → 텍스트 + 문서 구조

세제개편안 파일에서 글자와 정책 제목·표·적용시기를 읽어 들였습니다.

② 나누기 → 여러 개의 Chunk

문서 구조를 따라, 문맥이 끊기지 않는 조각으로 나누었습니다.

③ 꼬리표 · ④ 숫자로 (Chunk에서 갈라지는 두 갈래)

조각마다 신원(Metadata)을 연결하고, 조각 텍스트를 Vector로 바꿨습니다.

⑤ 저장 → 검색용 저장소

원문 · Vector · Metadata 를 연결해 한 건씩 담았습니다.

오늘 배운 다섯 단계는 왼쪽 아래 「Indexing」 화살표 하나 안에서 일어나는 일입니다.

다음 → 문서는 준비되었습니다. 그럼 이제 무엇이 남았을까요?

Chunk 에서 갈라지는 두 갈래

문서는 준비되었습니다. 그 다음은?

문서는 검색할 수 있는 상태로 준비되었습니다.

① 읽기

텍스트 + 문서 구조

② 나누기

여러 개의 Chunk

③ 꼬리표

Chunk + Metadata

④ 숫자로

Vector

⑤ 저장

검색용 저장소

그런데 사용자가 질문하면, 이 많은 Chunk 중 필요한 것을 어떻게 찾을까요?

다음 교안 → 02-2. RAG 질문검색과 문서선별

질문도 같은 방법으로 숫자로 바꾸고, 저장해 둔 Vector와 비교해서 관련 있는 조각을 골라오는 과정을 다룹니다.

오늘 다루지 않은 것

- 질문과 조각을 비교하는 방법
- 관련성 순위를 매기는 방법
- 몇 개를 골라올지 정하는 방법

오늘 준비해 둔 Chunk · Vector · Metadata 가 다음 시간의 출발점이 됩니다.